The lab exercises contain three primary C code examples demonstrating process creation, linking, and loading.

1. Process Creation using fork() (Exercise 1)

This program uses the fork() system call to create a new process.

Code Example

```
#include <stdio.h>

#include <unistd.h>


int main() {
   pid_t pid = fork();


   if (pid == 0) {
      printf("This is the child process. PID: %d\n", getpid());
   } else if (pid > 0) {
      printf("This is the parent process. PID: %d\n", getpid());
   } else {
      printf("Fork failed!\n");
   }


   return 0;
}
```

Explanation

pid_t pid = fork();: The fork() function attempts to create a new process, which is an exact copy of the calling process (the parent).

Return Value:

pid == 0: This condition is true only in the newly created child process. getpid() returns the child's Process ID (PID).

pid > 0: This condition is true only in the original parent process. The value of pid is the Process ID (PID) of the child process it just created. getpid() returns the parent's PID.

pid < 0: This indicates that the fork() call failed.

2. Role of the Linker (Exercise 5)

This example demonstrates how a program can be split into two separate source files, which are then compiled and linked together.

File 1: "file1.c"

```
#include <stdio.h>

Void hello() {
   Printf(“Hello from file1!\n”);
}
```

File 2: “file2.c”

```
Void hello();

Int main() {
   Hello();
   Return 0;
}
```

Explanation

file1.c defines the function hello().

file2.c declares the function hello() and calls it from main().

The compilation command gcc file1.c file2.c -o output_program compiles both files and, crucially, uses the Linker to resolve the call to hello() in file2.c with the actual definition of hello() found in file1.c (or its object code). The result is a single executable program.

3. Role of the Loader (Exercise 6)

This simple program is used to inspect the shared libraries it relies on, demonstrating the Loader's role at runtime.

Simple Program: "simple_program.c"

```
#include <stdio.h>

Int main() {
   Printf(“This is a simple program.\n”);
   Return 0;
}
```

Explanation

The program uses the printf() function, which is typically found in an external shared library (like libc).

The ldd command

(ldd simple_program) is used to list the dynamic dependencies (shared libraries) that the program requires when it runs.

The Loader’s job is to take this executable file, see the libraries it needs (as listed by ldd), and load those libraries into memory along with the program’s code and data when the program is executed.

Job of the Linker

The Linker is a program that takes one or more object files (the output of a compiler/translator) and combines them into a single executable file or library.

Symbol Resolution: Its main job is to resolve symbols (names of functions and variables).

For example, in the process of creating the output_program, the Linker finds the definition of the hello() function in the object code of file1.c and connects it to the call made in the object code of file2.c.

Relocation: It also organizes the code and data sections of the different input files so that they can be correctly addressed in the final output file.

Output: The Linker produces the Executable file.

Job of the Loader

The Loader is a part of the operating system that prepares a program for execution and executes it.

It is the final stage in the process of turning a source code file into a running program.

Program Loading: The Loader's primary job is to load the executable file from secondary storage (like a hard drive) into the main memory (RAM) of the computer.

Dynamic Linking: It is responsible for finding and loading any required dynamic (shared) libraries (like the ones identified by ldd) and making sure the program can access their functions.

Setup: It performs necessary setup, such as setting up the program's stack and heap, and then starts the program's execution by jumping to the program's entry point (e.g., the main function).